OPEN SOURCE · v0.5 · ALPHA

Contextual Memory Architecture (CMA)

A memory layer your agent carries with it.

The closed loop.

Three sentences. The entire thesis.

→ Memory becomes context.

→ Context shapes action.

→ **Action becomes memory.**

// the loop CMA implements

Stateless agents don't compound.

Every session starts cold. Experience never transfers.

01

Forget prior decisions

Every session re-litigates the same architectural choices.

02

Re-learn project context

Conventions, constraints, and quirks rediscovered on every task.

03

Repeat the same mistakes

Postmortems exist but never surface when the same failure recurs.

04

Lose relational context

RAG retrieves chunks, not the decisions linked to the project.

RAG is not memory.

The most useful context is structurally nearest, not semantically nearest.

STANDARD RAG RETRIEVES

- the nearest vector chunk
- by cosine similarity
- ignores edges, time, status
- indifferent to provenance

AGENT MEMORY NEEDS

→ decisions linked to projects

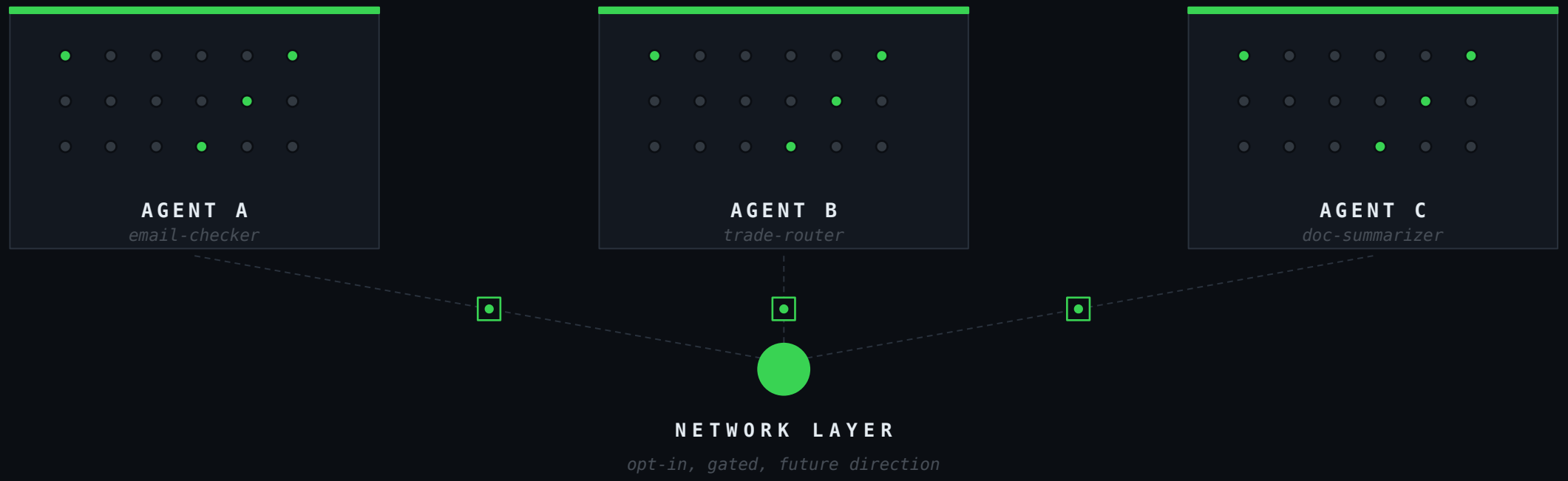
→ postmortems linked to failures

→ patterns inferred across tasks

→ supersession-aware exclusion

Per-agent. Not one shared brain.

Each agent owns its own vault. Cross-agent retrieval is opt-in, never default.



// fractal by design — same primitives compose upward. cross-agent edges require explicit policy.

Three intelligent functions.

Reasoner frames. Retriever recalls. Recorder forms.

01

REASONER

frames the goal,
decides what context matters

→

02

RETRIEVER

memory → context spec
via hybrid + graph search

→

03

RECORDER

experience → structured
durable memory

// memory feeds the next task

The vault is the memory.

Just markdown. Open in Obsidian. Version with git. Grep from the shell.

async-capital-call-processing.md

```
---
type: decision
title: Async Capital Call Processing
status: accepted
confidence: 0.86
tags: [capital-calls, performance, backend]
---
# Async Capital Call Processing

We decided to move capital call processing
into an async queue.

Uses [[Queue Retry Pattern]].
```

YAML frontmatter

machine-readable metadata

[[Wikilinks]]

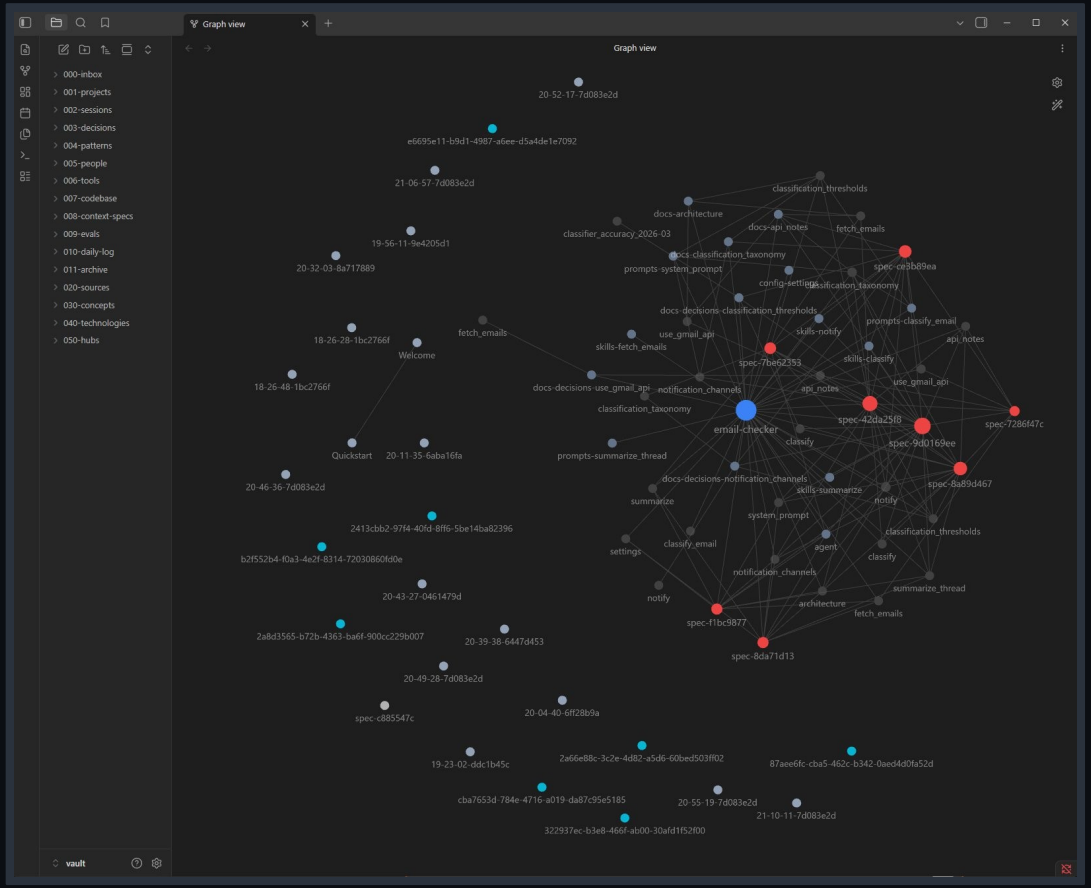
graph edges between notes

Filename

stable record id

Live from a working vault.

An email-checker agent. Every dot, a memory; every line, a relationship.



Obsidian graph view · email-checker agent · spec → source → backlink

Query to context, in five stages.

Deterministic. Inspectable. Reproducible.

01 **Hybrid seed search**

BM25L lexical + optional dense embeddings = hybrid score

02 **Beam-pruned multi-hop traversal**

Expand outward over wikilinks, beam-prune at each depth

03 **Final scoring**

metadata_boost × depth_decay ^ depth

04 **Paragraph-level fragment extract**

Score paragraphs against query, dedupe across nodes

05 **Context Spec assembly**

Structured artifact: fragments + relationship map + provenance

Hybrid scoring, depth-decayed.

Lexical and semantic combined, then modulated by metadata and depth.

$$\text{node_score} = \alpha \cdot \text{semantic} + (1 - \alpha) \cdot \text{lexical}$$

$$\text{final_score} = \text{node_score} \cdot \text{metadata_boost} \cdot \text{decay} \wedge \text{depth}$$

DEFAULTS

α	0.7
decay	0.80
beam_width	5
max_depth	2

METADATA BOOSTS

multiplicative, clamped at 0

accepted decision	× 1.10
rejected decision	× 0.70
superseded decision	× 0.50
archived (any type)	× 0.80
human_verified	× 1.10
confidence ≥ 0.85	× 1.08
confidence < 0.30	× 0.90

DEPTH DECAY

traversal penalty: decay ^ depth

DEPTH	HOPS	MULTIPLIER
0	direct match	× 1.00
1	one hop away	× 0.80
2	two hops away	× 0.64

// farther hops earn inclusion via topology

Confidence-gated memory writes.

Every potential write runs through these gates. The vault only sees what survives.

DECISION FLOW

confidence < 0.25 ?	yes →	**SKIP**	*dropped*
no			
status = accepted | rejected ?	yes →	**WRITE**	*→ vault*
no			
confidence ≥ 0.75 ?	yes →	**WRITE**	*→ vault*
no			
confidence ≥ 0.50 ?	yes →	**DRAFT**	*→ vault (draft)*
no			
otherwise	yes →	**PROPOSE**	*→ proposals/*

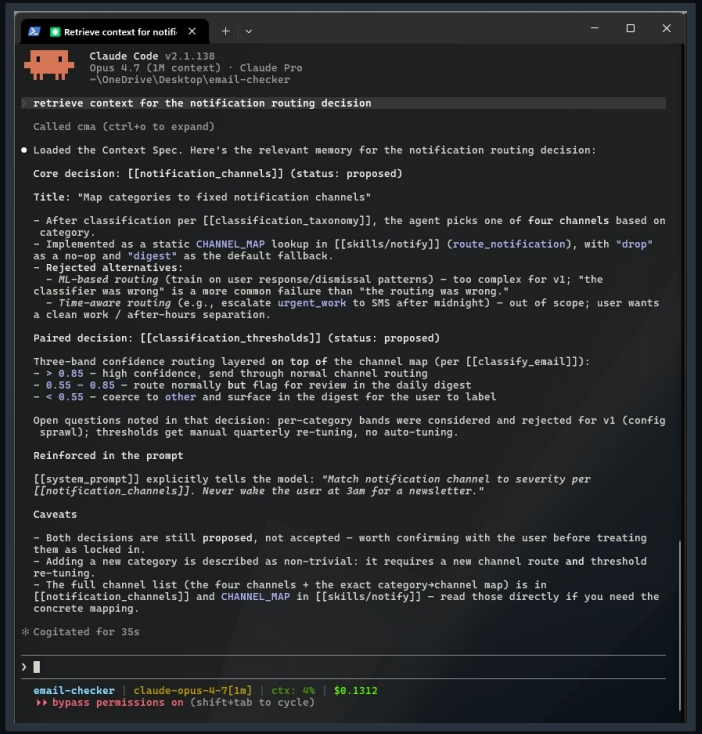
REFERENCE TABLE

STATUS	CONFIDENCE	ACTION
accepted	any	WRITE
rejected	any	WRITE
proposed	≥ 0.75	WRITE
proposed	0.50 – 0.74	DRAFT
proposed	0.25 – 0.49	PROPOSE
any	< 0.25	SKIP

// without gating, every guess lands in memory. compounding noise degrades reasoning.

What an agent sees.

A retrieve() call surfaces the relevant decision — linked, contextualized, ready to reason over.



OBSERVATION

Structured retrieval

Claude calls cma, gets a Context Spec — not a chunk dump.

Decisions surface as decisions

[[notification_channels]] arrives with status, rationale, and rejected alternatives.

Caveats included

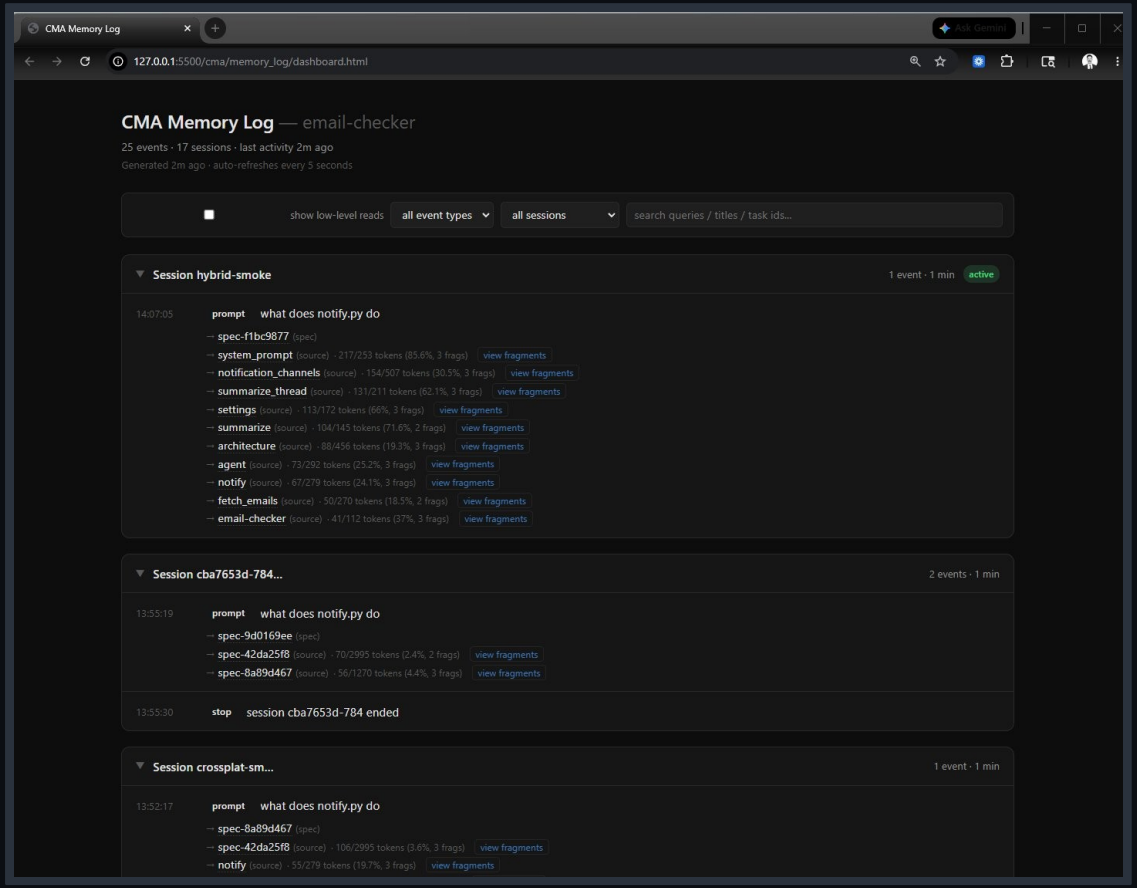
"Both decisions are still proposed, not accepted" — the agent knows what's tentative.

Self-narrating

What was loaded, why it was relevant, what to read next.

Memory you can watch.

Every retrieve, search, and write — logged, grouped by session, linked to every artifact.



cma/memory_log/dashboard.html · static, self-refreshing, no server

Ten MCP tools. Any agent.

Six graph primitives, four orchestrators. Stdio transport.

GRAPH PRIMITIVES

```
search_notes(query, top_k)
get_note(title)
get_outgoing_links(title)
get_backlinks(title)
traverse_graph(start, depth)
search_by_frontmatter(key, value)
```

ORCHESTRATORS

```
retrieve(query, max_depth, beam_width)
record_completion(yaml, dry_run)
graph_health()
reindex()
```

// wires into Claude Code via .mcp.json

Designed for single-CPU scale.

Footprint and warm-process latency on commodity hardware (MiniLM-L6-v2 384d).

VAULT SIZE	MARKDOWN	EMBEDDINGS	PROCESS RAM	WARM RETRIEVE	SOURCE
1K notes	~4 MB	1.5 MB	~750 MB	~40 ms	*measured*
10K notes	~37 MB	15 MB	~1.2 GB	~170 ms	*measured*
100K notes	~370 MB	~150 MB	~2 GB	~1.7 s	*projected*
1M notes	~3.7 GB	~1.5 GB	~10+ GB	**needs ANN**	*projected*

// 1K and 10K rows measured via docs/bench_scaling.py. 100K row extrapolated linearly from the 10K measurement.

// process RAM includes the ~500 MB resident sentence-transformers model. beyond ~100K, swap to FAISS / hnswlib.

// warm retrieve = in-process (MCP server case). shell-invoked cma retrieve pays a ~10-15 s model load per call.

Smarter agents, fewer tokens.

Same mechanism does both: cherry-pick fragments, persist what was learned.

→ **MEMORY COMPOUNDS**

Decisions, patterns, and postmortems from prior sessions surface automatically on the next relevant task. The agent inherits its own work.

→ **CONTEXT IS CHERRY-PICKED**

Fragment-level extraction, not whole-document dumping. The Retriever ranks paragraphs against the query and pulls only what's relevant.

DEMO MEASUREMENT

email-checker · one retrieve · 10 sources

Source bodies (total)	2,697 tokens
Fragments extracted	1,038 tokens
Reduction	**61.5 %**

ARCHITECTURAL BOUND

Output size is bounded by configuration (beam_width, max_depth, max_fragments_per_node), not by vault size. Cost per query stays flat as memory grows.

// measurement from cma/memory_log/activity.jsonl on the email-checker demo. your mileage will vary with vault and query.

Get started.

Local-first. Open source. Drop-in for any MCP-aware agent.

zsh

```
$  git clone github.com/danny-watkins/contextual-memory-architecture
$  cd contextual-memory-architecture
$  pip install -e ".[all]"
$  cd /path/to/your-agent-project
$  cma add                    # scaffold + ingest + wire MCP
$  claude                     # CMA fires on every prompt
```

→ **github.com/danny-watkins/contextual-memory-architecture**

Open source · Local-first · Built for Claude Code · Works with any MCP-aware agent